

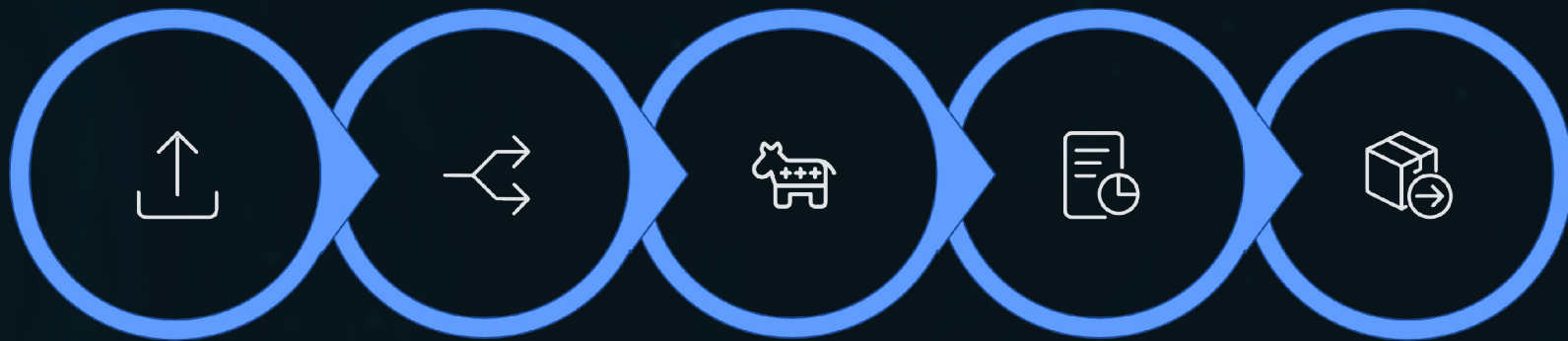
Brief AI 시스템 개발 프로젝트

1조

김상수 김효영 김의수 김진우



프로젝트 개요 : 파일 업로드 부터 AI 요약까지



파일 업로드

텍스트/이미지
분리

OCR 추출

AI 요약

결과 제공

다양한 문서 형식(PDF, Word, 이미지 등)을 업로드하면 텍스트와 이미지를 자동으로 구분하고, OCR로 파일내 텍스트를 인식 후 LLM 모델을 통해 생성된 통합 요약 결과를 제공

프로젝트 역할

김상수

Project Reader / Frontend

• 총괄 및 설계

- 프로젝트 관리 및 일정 수립
- 요구사항 정의 및 화면 흐름 설계
- 시스템 설계 및 환경 구성(Docker)

• Frontend 개발

- React/Vite 기반 UI 환경 구축
- 파일 업로드 및 다운로드 구현
- 요약 결과 및 이력 확인

인터페이스

김효영

Backend Developer

• API 개발

- FastAPI 기반 서버 아키텍처 구축
- 파일 수신 및 데이터 생성

• 데이터 연동

- PostgreSQL 연동 및 저장
- 업로드 파일 형식 체크 및 용량 체크

- JSON 결과 응답 처리

김의수, 김진우

AI & Data Engineering

• 총괄 및 설계

- OCR 파싱(PyMuPDF, PaddleOCR)
- 텍스트 전처리 및 텍스트 데이터

추출

• LLM 연동

- Ollama(Gemma3:4b) 로컬 LLM 연동

- 프롬프트 엔지니어링(요약/분류)

개발 스펙

파일 업로드를 통한 AI 요약까지 프로젝트에 사용된 핵심 기술 구성

Frontend

React 기반 웹 애플리케이션

React

Vite

Typescript

Axios

Backend

FastAPI 기반 API 서버

Python

FastAPI

SQLAlchemy

Uvicorn

AI / OCR

생성형 AI 및 문서 인식

Ollama

Gemma

PyMuPDF

PaddleOCR

Database

PostgreSQL 기반 데이터 관리

PostgreSQL

SQLAlchemy

Infra

Docker 기반 공통 환경 구성

Docker

Docker
compose

Container

CI/CD

Integration

시스템 통합 및 연동

RestFul

Websocket

Config

핵심 기능 및 기술

01

회원가입/로그인/로그아웃

사용자별 요약된 요청 문서 구분을 위한 확인

02

파일 업로드(Drag & Drop)

파일 업로드 기능을 통한 내용 요약 요청

03

요약 형태 설정

사용자 니즈에 맞춘 형태(간략히, 기본, 상세히)로 요약 결과 제공

04

이미지 OCR(PyMuPDF, PaddleOCR)

OCR의 활용으로 다양한 형태의 파일에서 가장 효율적인 방법으로 텍스트 추출

05

파일 요약 이력 확인

사이드바에서 요약 요청한 파일에 대한 이력 확인

06

파일 다운로드(TXT, PDF)

요약된 결과를 텍스트나 PDF파일 형식으로 다운로드

07

텍스트 / 이미지 분리

텍스트·이미지 분리 후 OCR 기반 텍스트 추출 및 재조립

08

AI 요약

Ollama가 Gemma 3 모델을 구동해 핵심 내용을 요약

개발 일정

WBS 기반 타임라인 마일스톤

05.20
화

05.21
수

05.22
목

05.23
금

05.24
토

05.25
일

05.26
월

05.27
화

기획

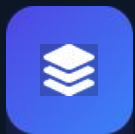
환경 구축

개발

테스트

배포

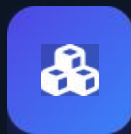
Frontend 시스템 아키텍처



코어 스택

Foundation

React + Vite + TypeScript 기반의 초고속
빌드 및 타입 안정성을 확보한 SPA 환경 구축



모듈화된 디자인

Component

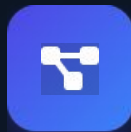
Atomic Design 및 컴포넌트 단위 설계를 통해
코드 재사용성을 극대화 및 유지보수 효율
증가



효과적인 데이터 흐름

Axios

Axios 인터셉터 및 커스텀 훅을 활용한
체계적인 API 통신으로 데이터 정합성과
예외처리 관리



분리형 구조

Interface

UI 로직과 비즈니스/백엔드 로직을 엄격히
분리하여 독립적인 개발 및 테스트가 가능한
구조 지향

회원가입 / 로그인 및 사용자 인증

🛡️ 회원 인증 UI (Auth)

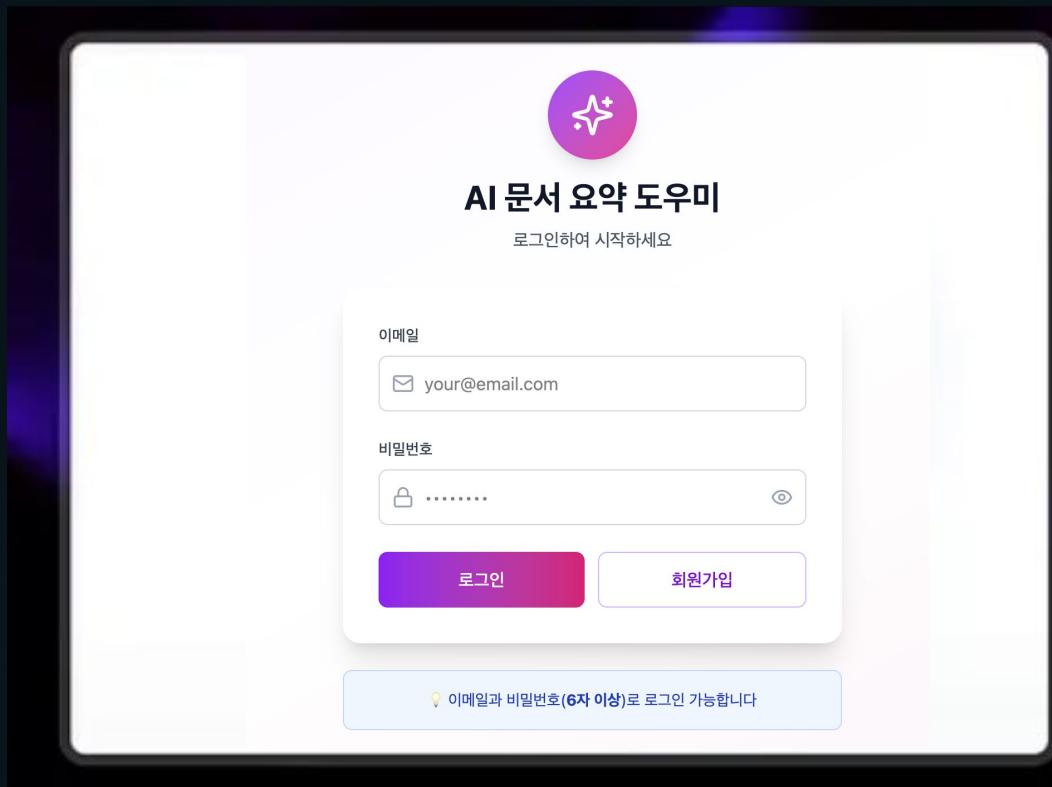
이메일 및 비밀번호(6자 이상) 기반의 직관적인 로그인/회원가입 인터페이스를 통해 개인별 맞춤 환경 제공

🖥️ 실시간 상태 인터랙션

문서 업로드부터 AI 분석 완료까지의 진행 상태를 실시간으로 가시화하여 사용자의 체감 대기 시간을 단축합니다.

🔧 사용자 중심 컴포넌트

React 기반의 컴포넌트 단위 설계를 통해 일관된 UI 톤앤매너를 유지하며 최적의 사용성을 확보했습니다.



The image shows a user interface for an AI document summarization service. At the top, there is a purple circular logo with a white star and plus sign. Below it, the text "AI 문서 요약 도우미" (AI Document Summary Assistant) is displayed in bold, followed by "로그인하여 시작하세요" (Log in to get started). The main form area contains two input fields: "이메일" (Email) with the placeholder "your@email.com" and "비밀번호" (Password) with a masked input "....." and an eye icon for toggling visibility. Below these fields are two buttons: a purple "로그인" (Login) button and a white "회원가입" (Sign Up) button. At the bottom, a light blue banner contains the text "📍 이메일과 비밀번호(6자 이상)로 로그인 가능합니다" (You can log in with email and password (6 characters or more)).

Frontend 메인화면 주요 기능



회원 인증 및 프로필

로그인 후 우측 상단 사용자 인증 상태 가시화 및 계정 관리



Drag & Drop 업로드

사용자 편의를 극대화한 직관적인 파일 드래그 방식 지원



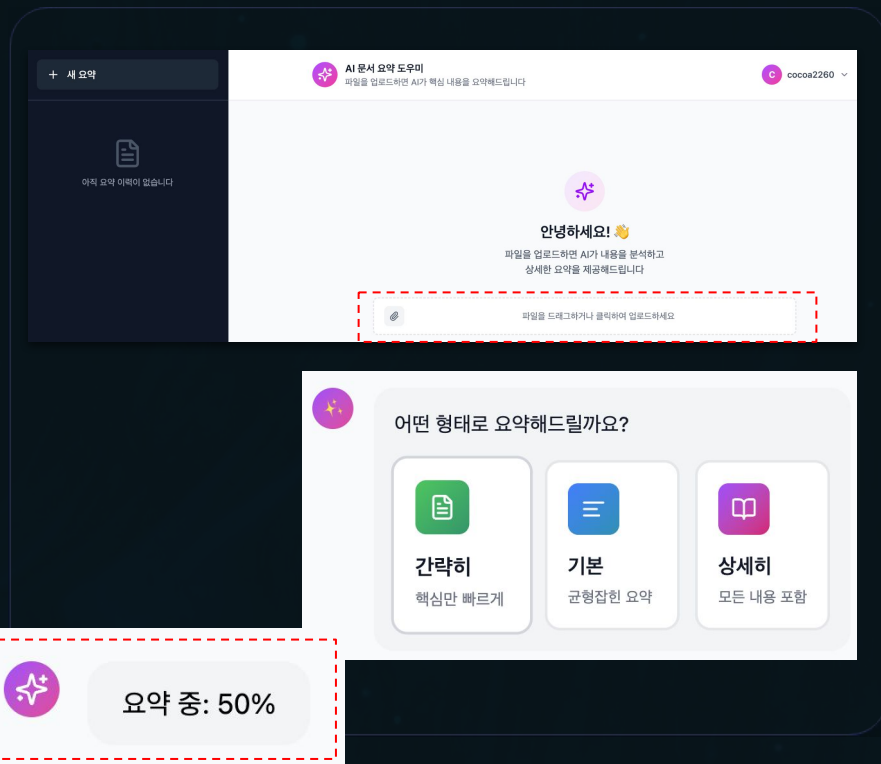
요약 옵션 커스터마이징

간략히, 기본, 상세히 등 3단계 요약 강도 개인별 설정



실시간 상태 모니터링

AI 분석 및 요약 진행률 실시간 트래킹 및 가시화



체계적인 데이터 및 이력 관리

+ 새 요약

보고서 양식 (3).pdf

차세대 신제품 기획 프로젝트팀 구성 및 진행 사항에 대한 논의입니다.....

🕒 5월 26일 오후 05:14

 **AI 문서 요약 도우미**
파일을 업로드하면 AI가 핵심 내용을 요약해드립니다

C cocoa2260

👤

보고서 양식 (3).pdf (204.9 KB)

👤

차세대 신제품 기획 프로젝트팀 구성 및 진행사항에 대한 논의입니다.

⋮

📄 복사하기

📄 TXT 다운로드

📄 PDF 다운로드

사이드바 이력 관리

좌측 사이드바를 통해 과거에 요약했던 문서 리스트를 실시간으로 확인하고 즉시 다시 열람할 수 있는 시스템입니다.

요약 형태 커스터마이징

사용자의 니즈에 따라 간략히, 기본, 상세히 등 요약의 깊이를 직접 설정하여 최적화된 결과물을 도출합니다.

텍스트 / 이미지 자동 분리

문서 내의 일반 텍스트와 이미지 영역을 자동으로 구분하여 OCR 기술의 효율을 극대화하는 분석 프로세스입니다.

ERD 설계

유저 테이블의 user_ID를 외래키로 참조하여 어떤 사용자의 문서인지 식별

USER_INFO TABLE		
컬럼명 (Column)	타입 (Type)	설명
USER_ID(PK)	UUID	사용자를 식별하는 고유 ID
EMAIL	String (Unique)	사용자 이메일 (로그인 ID)
PASSWORD_HASH	String	비밀번호 해시값
ROLE	String	역할 (Admin, User)
CREATE_AT	DateTime	계정 생성 시간
UPDATE_AT	DateTime	정보 수정 시간
IS_ACTIVE	Boolean	계정 활성 상태 (true, false)

Enum (CATEGORY) 값 예시

- INVOICE : 송장
- CONTRACT : 계약서
- REPORT : 보고서

범례

PK Primary Key (기본 키)

FK Foreign Key (외래 키)

DOCUMENT_RECORDS TABLE		
컬럼명 (Column)	타입 (Type)	설명
ID	UUID	history ID
USER_ID(FK-USERS.USER_ID)	UUID	사용자 ID
FILE_ID(PK)	UUID	고유 ID
FILE_NAME	String	사용자가 업로드한 원본 파일명
CATEGORY	Enum	문서 분류 종류
SUMMARY	Text	요약 문서 내용
UPLOAD_AT	DateTime	파일 업로드 시간
PROCESS_AT	DateTime	AI 분석 완료 시간
TASK_STATUS	String	문서 요약 상태

1 — ∞ 여러 개 (1:N 관계)

FastAPI Flow

id 유무에 따라 '상세 조회'와 '서브쿼리 그룹화를 통한 목록 조회'



Websocket - 작업 상태 조회

```
await manager.send(file_id,{  
    "percent": 10,  
    "state" : "PROGRESS"  
})
```

```
await asyncio.sleep(1)
```

```
# ocr 시작  
extracted_text = await run_in_threadpool(  
    process_pdf,  
    file_bytes  
)
```

```
await manager.send(file_id,{  
    "percent": 40,  
    "state" : "PROGRESS",  
    "extracted_text" : extracted_text  
})
```

```
await asyncio.sleep(1)
```

```
await manager.send(file_id,{  
    "percent": 50,  
    "state" : "PROGRESS",  
})
```

```
# llm 시작  
llm_result= await run_in_threadpool(subtract_text,extracted_text,check_length)  
llm_result = json.loads(llm_result)  
category=llm_result["category"]  
summary = llm_result["summary"]
```

```
await manager.send(file_id,{  
    "percent": 90,  
    "state" : "PROGRESS",  
})
```

10%

OCR 프로세스 준비

Backend worker 할당 및 OCR 엔진 초기화

40%

OCR 완료

50%

LLM 프로세스
준비

90%

AI 인사이트 도출 완료

LLM을 통한 요약 및 카테고리 분류 처리 완료

100%

최종 데이터 전송

요약(Summary) 및 카테고리(Category) 페이로드 전달

Asyncio - 비동기 처리

(연산이 무겁고 오래걸리는 작업인 LLM, OCR을 백그라운드 태스크로 분리)



파일 업로드 후 백그라운드 태스크에서 OCR, LLM 연산이 수행되는 동안에도, 서버가 블로킹되지 않고 다운로드 API 등 다른 요청을 동시에 처리할 수 있는 환경을 구축

사용 라이브러리

PyMuPDF · PaddleOCR

PyMuPDF

문서 페이지에서 텍스트 및 이미지를
추출하는 라이브러리. 기존 PyPDF 대비
성능과 정확도가 크게 개선되었습니다.

PaddleOCR

문서에서 추출된 이미지 내 텍스트를
인식하기 위한 OCR 라이브러리.

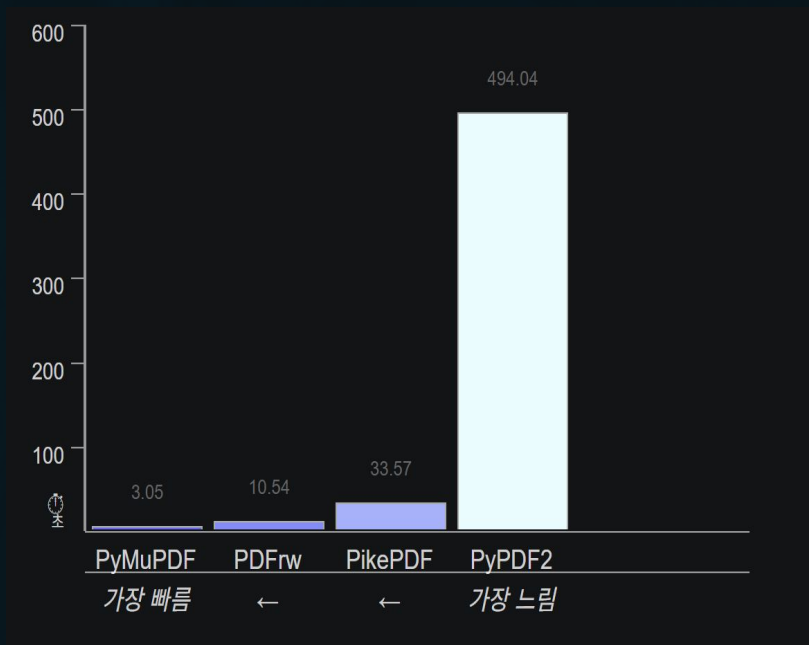
```
1 import os
2 import sys
3 from pathlib import Path
4 from datetime import datetime
5 import json
6 import logging
7
8 ROOT_DIR = Path(__file__).resolve().parent.parent
9 LOG_FILE = ROOT_DIR / "logs" / "app.log"
10
11 def get_logger(name: str) -> logging.Logger:
12     logger = logging.getLogger(name)
13     logger.setLevel(logging.INFO)
14     if not logger.handlers:
15         handler = logging.StreamHandler(sys.stdout)
16         formatter = logging.Formatter(
17             "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
18         )
19         handler.setFormatter(formatter)
20         logger.addHandler(handler)
21     return logger
22
23 def load_config(path: Path) -> dict:
24     if not path.exists():
25         raise FileNotFoundError(f"Config not found: {path}")
26     with path.open("r", encoding="utf-8") as f:
27         return json.load(f)
28
29 def save_config(config: dict, path: Path) -> None:
30     with path.open("w", encoding="utf-8") as f:
```

PyMuPDF, PyPDF2 성능 비교1

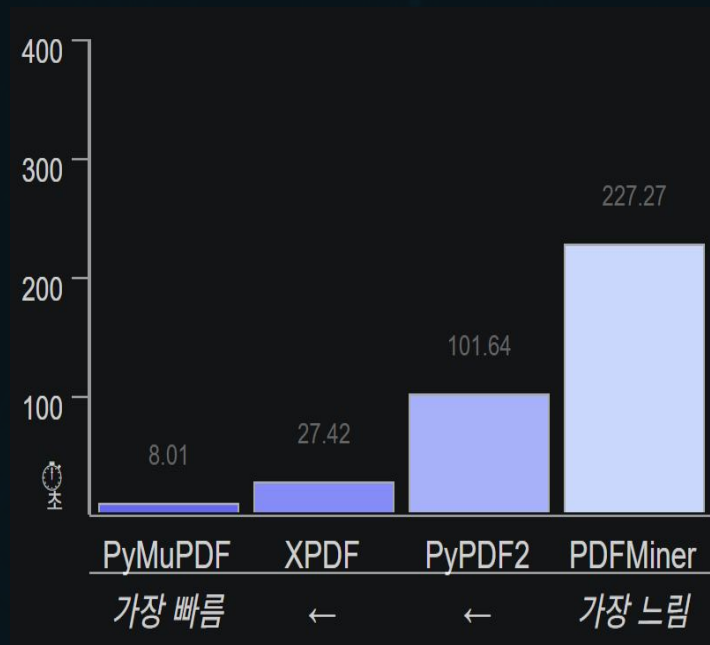
라이브러리 성능 비교

라이브러리	PyMuPDF	PyPDF2	pdfcrowd
텍스트 추출	모든 문서 유형	pdf	pdf
다중 지원 문서 형식	pdf, txt, 이미지, svg, docs, xlsx, pptx, hwp	pdf	pdf
추출 속도	빠름	느림	보통

PyMuPDF, PyPDF2 성능 비교2



복사



텍스트 추출

PaddleOCR과 Block

```
if ocr_instance is None:
    from paddleocr import PaddleOCR

    # PaddleOCR 3.x 계열 기준 옵션
    # 사용 중인 버전에 따라 일부 옵션명이 다를 수 있다.
    ocr_instance = PaddleOCR(
        lang="korean",
        device="cpu",

        # 문서 방향 분류 비활성화
        # PDF 슬라이드 OCR에서는 보통 필요하지 않고,
        # 커두면 속도가 느려질 수 있다.
        use_doc_orientation_classify=False,

        # 문서 펼침 / 왜곡 보정 비활성화
        # 일반 PDF 이미지 영역에서는 필요 없는 경우가 많다.
        use_doc_unwarping=False,

        # 텍스트 라인 방향 감지 비활성화
        # 네 환경에서 OCR 오류가 발생했기 때문에 안정성 우선으로 끈다.
        use_textline_orientation=False,
    )

return ocr_instance
```

PaddleOCR

이미지 안에 텍스트를 인식하기 위해 사용했습니다.
예) PDF 내 이미지나, 표는 PyMuPDF만으로는
이미지 상에 있는 텍스트를 추출하기 어려워 이미지
영역만 잘라 PaddleOCR로 텍스트를 인식하도록
했습니다.

Block

type을 통해 image block과 text block으로 타입을
구분하여 이미지 영역을 ocr 하게끔 지정하고 텍스트
영역을 바로 추출하여 합치도록 코드를 작성했습니다.

```

'format': {
  'type': 'object',
  'properties': {
    'summary': {
      'type': 'string',
    },
    'category': {
      'type': 'string',
      'enum': [
        '기술/개발문서',
        '법률/판례',
        '기획안/제안서',
        '경영/비즈니스',
        '교육/학술',
        '행정/공공문서',
        '생활/가정',
        '금융/회계',
        '의료/건강',
        '기타/미분류'
      ]
    }
  },
  'required': ['summary', 'category']
},

```

OUTPUT FORMAT

- 요약 결과와 카테고리를 분류해서 출력
- 카테고리는 enum으로 결과 값 제한



prompt에서 작업 과정을 나눈 뒤 각 작업에 대한 RULE 추가

[TASK]

Step 1. Document Understanding (CRITICAL STEP)

Step 2. Summarization

[Summary RULE]

Step 3. Classification (STRICT RULE)

[CRITICAL INSTRUCTION]

[ROLE → CATEGORY MAPPING RULE]

요약 형태 선택 프롬프트

```
BASE_PROMPT = '''
```

```
...  
{length_prompt}  
...  
'''
```

length_prompt : 파라미터로 전달된 값



간략히

핵심만 빠르게

```
'SHORT': '''
```

```
Describe ONLY the primary purpose of the document.
```

- Focus on why the document exists.
- Do NOT summarize contents.
- Do NOT mention implementation, technologies, or detailed functions.
- Maximum 1 sentence.

```
'''
```



기본

균형잡힌 요약

```
'MIDDLE': '''
```

```
Summarize the document contents and major functions.
```

```
Include:
```

- main topic
 - key functions
 - important contents
- ```
Keep it concise.
```

```
'''
```



상세히

모든 내용 포함

```
'LONG': '''
```

```
Provide a detailed and comprehensive explanation of the document.
```

```
You MUST include:
```

- major topics and subtopics
- important details and supporting information
- key processes, structures, or relationships
- main functions, roles, or responsibilities
- relevant context and background
- important entities, organizations, or environments
- policies, methods, or procedures if mentioned
- examples, conditions, or operational aspects if available

```
'''
```

## AI 모델 파라미터 조정

```
'options': {
 'num_ctx': 32768,
 'num_batch': 512,

 'temperature': 0.1,
 'top_k': 20,
 'top_p': 0.8,

 'num_predict': 2048,
```

**num\_ctx** : 모델이 한번에 참조할 수 있는 컨텍스트의 크기

Default : 2048 (token)



32768 (token)

**temperature** : 모델 출력의 무작위성(창의성) 조절  
→ 높을수록 랜덤성 증가

Default : 0.8



0.1

# 경량 AI 모델 비교

| AI Model                              | Gemma3 : 4b            | exaone3.5:2.4b         | Llama3.2:3b | Qwen2.5:3b   |
|---------------------------------------|------------------------|------------------------|-------------|--------------|
| 시나리오 1: 일반 문서 (11,576자, 18페이지 PDF)    |                        |                        |             |              |
| 시간                                    | 1분 이내                  | 1분 이내                  | 30초 이내      | 30초 이내       |
| 특징                                    | 한글-영어 혼합 안정적<br>정확한 요약 | 한글-영어 혼합 불안정<br>일관성 부족 | 한글 입/출력에 약함 | 한글 오타자 인식 약함 |
| 시나리오 2: 대용량 문서 (205,070자, 116페이지 PDF) |                        |                        |             |              |
| 시간                                    | 약 2분                   | 약 4분                   | 약 15분       | 오류 발생        |
| 특징                                    | 영어 출력                  | 불완전한 결과                |             | 요약 결과 : None |

## 소감

### 김상수

하나의 서비스를 완성해가는 과정에서 협업과 설계의 중요성을 느낄 수 있었고, 작은 기능 하나도 사용자 경험에 큰 영향을 줄 수 있다는 점을 배울 수 있었습니다.

### 김효영

비동기와 웹소켓으로 유저 경험과 데이터 무결성을 모두 경험해 볼 수 있었습니다. 또한, 설계에 대한 중요성을 깨닫게 되었습니다.

### 김진우

이번 프로젝트를 통해 단순히 모델을 사용하는 것을 넘어, 원하는 결과를 얻기 위해 프롬프트와 추론 흐름을 설계하는 과정의 중요성을 느낄 수 있었습니다.

### 김의수

OCR과 LLM을 단순히 사용하는 것이 아니라, 텍스트 추출과 프롬프트 설계를 직접 다뤄보며 AI 기반 문서 처리의 가능성과 한계를 함께 경험할 수 있었습니다.